Universidad de Guadalajara

Centro Universitario de Ciencias Exactas e Ingenierías

Ingeniería informática



Algoritmos metaheurísticos

REPORTE TÉCNICO: ACTIVIDAD 6 - EVOLUCIÓN DIFERENCIAL (DE)

Actividad 6 : Evolución Diferencial (Michalewicz y Six-Hump Camel)

Minimización: Funciones de Six-Hump Camel y Michalewicz

Ángel Jean Fernández Argoul

1. INTRODUCCIÓN

La **Evolución Diferencial (DE)** es un algoritmo estocástico de búsqueda directa propuesto por Storn y Price en 1997. A diferencia de los algoritmos genéticos tradicionales que utilizan cruzas probabilísticas basadas en la aptitud, DE perturba la población actual utilizando diferencias escaladas de vectores seleccionados aleatoriamente. Esta característica le otorga una excelente capacidad para explorar paisajes de optimización continuos, siendo muy resistente al estancamiento en mínimos locales sin requerir el uso de gradientes.

En este trabajo, el algoritmo DE se pone a prueba minimizando dos funciones matemáticas clásicas de alta complejidad: Michalewicz (reconocida por sus múltiples valles estrechos) y Six-Hump Camel (que posee seis mínimos locales, dos de ellos globales).

2. OBJETIVO

Implementar el algoritmo de Evolución Diferencial (DE) para encontrar el mínimo global de las funciones de prueba **Michalewicz** y **Six-Hump Camel**. Se documentará la convergencia del algoritmo y se representará gráficamente la evolución espacial de las poblaciones desde la inicial hasta la final.

3. MARCO TEÓRICO

El algoritmo DE opera sobre una población de NP vectores (individuos) de dimensión D. En cada generación, los individuos evolucionan a través de tres operadores principales:

3.1 Mutación (Diferencial)

Para cada vector objetivo Xi, se seleccionan aleatoriamente tres vectores distintos X_{r1}, X_{r2}, X_{r3} de la población. Se genera un **vector mutante** Vi sumando al primero la diferencia escalada de los otros dos:

$$Vi = X_{r1} + F \cdot (X_{r2} - X_{r3})$$

Donde $F \in [0,2]$ es el factor de mutación (escalamiento).

3.2 Cruza (Recombinación Binomial)

El vector mutante V_i se recombina con el vector objetivo original X_i para crear un **vector de prueba (trial)** U_i. Para cada dimensión j, se lanza un número aleatorio frente a la tasa de cruza $CR \in [0,1]$:

$$U_{i,j} = \begin{cases} V_{i,j} & \text{si } rand(0,1) \leq CR \text{ o } j = j_{rand} \\ X_{i,j} & \text{en caso contrario} \end{cases}$$

El índice j_{rand} garantiza que el vector de prueba herede al menos una dimensión del mutante.

3.3 Selección

Se evalúa la aptitud (*fitness*) del vector de prueba. Si es mejor o igual al original, lo reemplaza en la siguiente generación (selección codiciosa):

$$X_i^{gen+1} = \begin{cases} U_i & \text{si } f(U_i) \leq f(X_i) \\ X_i & \text{en caso contrario} \end{cases}$$

3.4 Funciones Objetivo

Las funciones a minimizar en esta actividad son:

1. Michalewicz (2D):

$$f(x) = -\sum_{I=1}^{2} sin(x_i) \sin^{20}\left(\frac{ix_i^2}{\pi}\right), \; x_i \in [0,\pi]$$

2. Six-Hump Camel Function:

$$f(x) = \left(4 - 2.1x_1^2 + \frac{x_1^4}{3}\right)x_1^2 + x_1x_2 + (4x_2^2 - 4)x_2^2, \; x1 \in [-3,3], \; x2 \in [-2,2]$$

4. DESARROLLO (CÓDIGO COMENTADO)

El algoritmo fue programado en C++. Se establecieron parámetros estándar empíricos: Población (NP=50), Factor de Mutación (F=0.8), y Tasa de Cruza (CR=0.9).

A continuación, se presenta el bloque central del algoritmo (`evolucionar`), donde se eligen los vectores aleatorios y se aplican la mutación, la cruza binomial y la selección de forma optimizada en el mismo ciclo:

```
for (int i = 0; i < n_part; i++) {

            // 1. Elegir r1, r2, r3 distintos entre si y distintos de i
```

```
            int r1, r2, r3;
            do { r1 = dist_int(motor); } while (r1 == i);
            do { r2 = dist_int(motor); } while (r2 == i || r2 == r1);
            do { r3 = dist_int(motor); } while (r3 == i || r3 == r1 || r3
== r2);


            // 2. Mutación y Cruza
            vector<double> u_prueba = poblacion[i];
            int j_rand = dist_dim(motor); // Asegura que herede al menos
una dimensión


            for (int d = 0; d < dims; d++) {
                if (dist_r(motor) <= CR || d == j_rand) {
                    double v_mutante = poblacion[r1][d] + F * (pobla-
cion[r2][d] - poblacion[r3][d]);
                    // Clamping a los limites
                    v_mutante = max(lim_inf[d], min(lim_sup[d], v_mu-
tante));
                    u_prueba[d] = v_mutante;
                }
            }

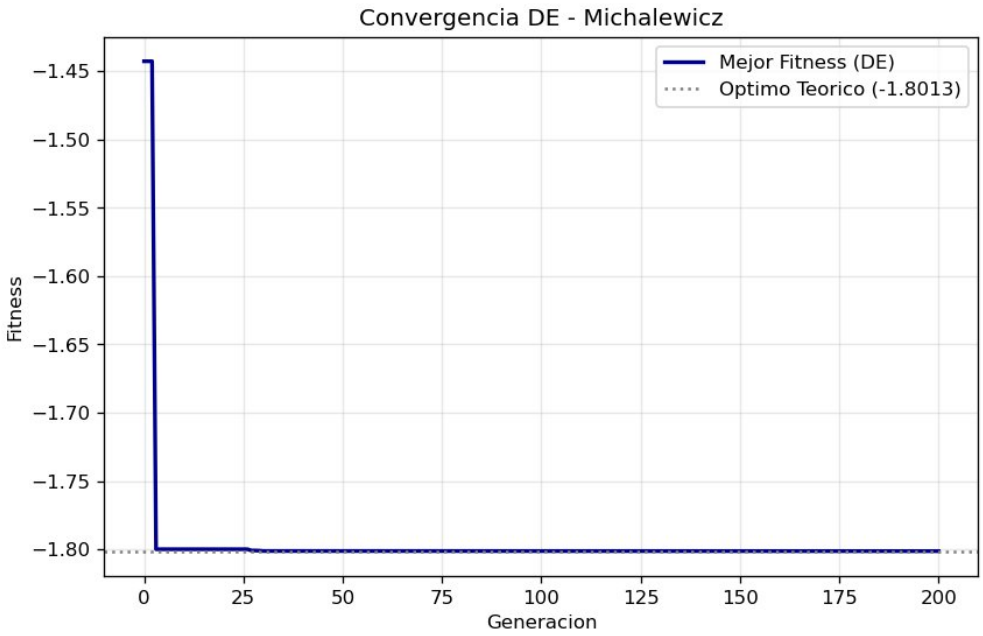

            // 3. Selección
            double fit_prueba = func_obj(u_prueba);
            if (fit_prueba <= fitness_pob[i]) {
                nueva_poblacion[i] = u_prueba;
                nueva_fitness[i] = fit_prueba;
```

```
                // Actualizar Global Best
                if (fit_prueba < gbest_val) {
                    gbest_val = fit_prueba;
                    gbest_pos = u_prueba;
                }
            }
        }


        poblacion = nueva_poblacion;
        fitness_pob = nueva_fitness;
    }
```

5. RESULTADOS

1. **Gráfica de Convergencia :** Documentan la minimización generación tras generación hasta alcanzar el mínimo teórico de −1.8013 para Michalewicz y −1.0316 para Six-Hump Camel.



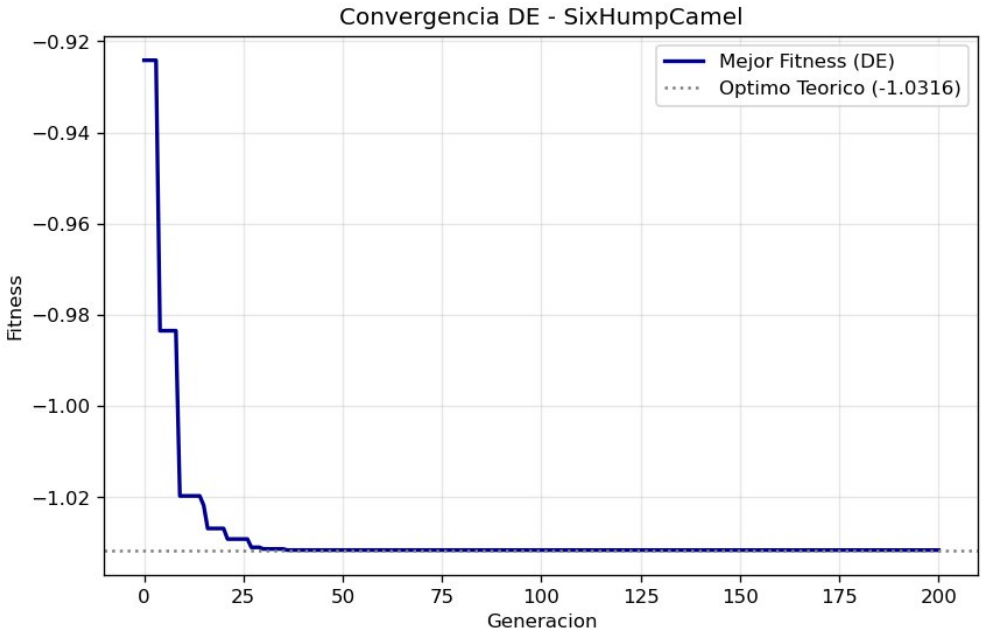
RESULTADOS

Optimo teorico:
-1.801300

Valor encontrado:
-1.801303

Diferencia:
0.000003

Gens: 200 | Parts: 50
F: 0.8 | CR: 0.9



RESULTADOS

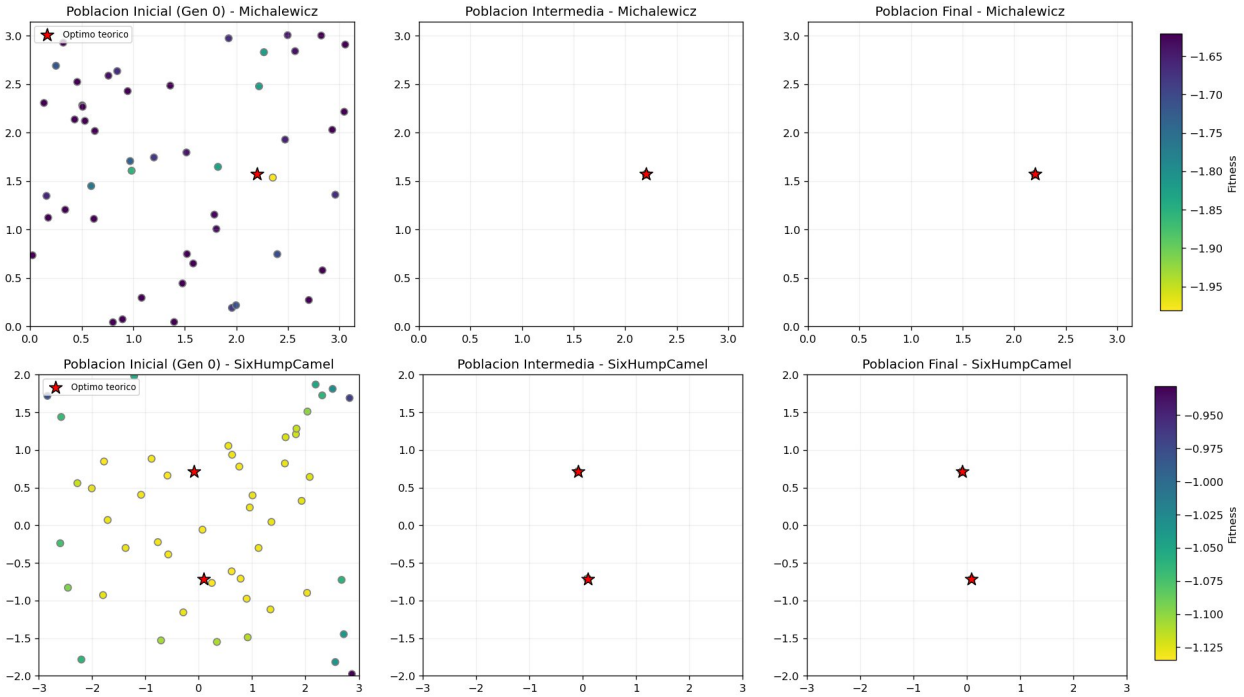
Optimo teorico:
-1.031600

Valor encontrado:
-1.031628

Diferencia:
0.000028

Gens: 200 | Parts: 50
F: 0.8 | CR: 0.9

2. Evolución Poblacional: Las gráficas de dispersión ilustran la transición desde un estado aleatorio inicial (Gen 0) hasta la convergencia absoluta en las coordenadas óptimas (Gen Final).


Poblacion Inicial (Gen 0) - Michalewicz
Optimo teorico
Poblacion Intermedia - Michalewicz
Poblacion Final - Michalewicz
Fitness
Poblacion Inicial (Gen 0) - SixHumpCamel
Optimo teorico
Poblacion Intermedia - SixHumpCamel
Poblacion Final - SixHumpCamel
Fitness

6. CONCLUSIONES

La implementación de la Evolución Diferencial (DE) confirmó su reputación como uno de los optimizadores continuos más robustos. A diferencia de técnicas basadas puramente en vecindarios estáticos, el uso de tres vectores aleatorios para la mutación (r1,r2,r3) garantizó un mantenimiento autónomo de la diversidad; al inicio, cuando la población estaba dispersa, las "diferencias" calculadas causaban saltos de exploración largos. Conforme la población convergía y se agrupaba cerca del óptimo global, los vectores se acortaban naturalmente, resultando en saltos de explotación más finos y precisos. Ambas funciones lograron evadir sus respectivos mínimos locales y alcanzaron el éxito numérico esperado en menos de 200 iteraciones.

7. Resto del código

C++ :

```cpp
#include <iostream>
#include <vector>
#include <cmath>
#include <random>
#include <algorithm>
#include <fstream>
#include <string>
#include <iomanip>

using namespace std;

const double PI = 3.141592653589793;
```

```cpp
// --- Funciones Objetivo ---

// 1. Michalewicz 2D (m=10 implícito en la potencia 20)
double michalewicz(const vector<double>& x) {
    double res = 0.0;
    for (int i = 0; i < 2; i++) {
        res -= sin(x[i]) * pow(sin((i + 1) * x[i] * x[i] / PI), 20);
    }
    return res;
}

// 2. Six-Hump Camel Function
double six_hump_camel(const vector<double>& x) {
    double x1 = x[0];
    double x2 = x[1];
    double term1 = (4.0 - 2.1 * x1 * x1 + pow(x1, 4.0) / 3.0) * x1 * x1;
    double term2 = x1 * x2;
    double term3 = (-4.0 + 4.0 * x2 * x2) * x2 * x2;
    return term1 + term2 + term3;
}

// --- Clase Differential Evolution (DE) ---
class DE {
private:
    int n_part, dims, iters;
    double F, CR;
    vector<double> lim_inf, lim_sup;
```

```
    vector<vector<double>> poblacion;
    vector<double> fitness_pob;
    mt19937 motor;

    vector<double> gbest_pos;
    double gbest_val;

    double (*func_obj)(const vector<double>&);
    string nombre_func;

public:
    DE(int np, int dimensiones, int iteraciones, vector<double> inf, vec-
tor<double> sup, double (*f)(const vector<double>&), string nombre) {
        n_part = np;
        dims = dimensiones;
        iters = iteraciones;
        lim_inf = inf;
        lim_sup = sup;
        func_obj = f;
        nombre_func = nombre;

        F = 0.8;  // Factor de Mutación [0, 2]
        CR = 0.9; // Tasa de Cruza [0, 1]

        gbest_val = numeric_limits<double>::max();
        random_device rd;
```

```
        motor = mt19937(rd());
    }

    void inicializar() {
        poblacion.resize(n_part, vector<double>(dims));
        fitness_pob.resize(n_part);

        for (int i = 0; i < n_part; i++) {
            for (int d = 0; d < dims; d++) {
                uniform_real_distribution<double> dist_pos(lim_inf[d],
lim_sup[d]);
                poblacion[i][d] = dist_pos(motor);
            }
            fitness_pob[i] = func_obj(poblacion[i]);

            if (fitness_pob[i] < gbest_val) {
                gbest_val = fitness_pob[i];
                gbest_pos = poblacion[i];
            }
        }
    }

    void evolucionar(int gen) {
        vector<vector<double>> nueva_poblacion = poblacion;
        vector<double> nueva_fitness = fitness_pob;
        uniform_real_distribution<double> dist_r(0.0, 1.0);
        uniform_int_distribution<int> dist_int(0, n_part - 1);
```

```
        uniform_int_distribution<int> dist_dim(0, dims - 1);


        for (int i = 0; i < n_part; i++) {
            // 1. Elegir r1, r2, r3 distintos entre si y distintos de i
            int r1, r2, r3;
            do { r1 = dist_int(motor); } while (r1 == i);
            do { r2 = dist_int(motor); } while (r2 == i || r2 == r1);
            do { r3 = dist_int(motor); } while (r3 == i || r3 == r1 || r3
== r2);


            // 2. Mutación y Cruza
            vector<double> u_prueba = poblacion[i];
            int j_rand = dist_dim(motor); // Asegura que herede al menos
una dimensión


            for (int d = 0; d < dims; d++) {
                if (dist_r(motor) <= CR || d == j_rand) {
                    double v_mutante = poblacion[r1][d] + F * (pobla-
cion[r2][d] - poblacion[r3][d]);
                    // Clamping a los limites
                    v_mutante = max(lim_inf[d], min(lim_sup[d], v_mu-
tante));
                    u_prueba[d] = v_mutante;
                }
            }


            // 3. Selección
            double fit_prueba = func_obj(u_prueba);
```

```
            if (fit_prueba <= fitness_pob[i]) {
                nueva_poblacion[i] = u_prueba;
                nueva_fitness[i] = fit_prueba;

                // Actualizar Global Best
                if (fit_prueba < gbest_val) {
                    gbest_val = fit_prueba;
                    gbest_pos = u_prueba;
                }
            }
        }

        poblacion = nueva_poblacion;
        fitness_pob = nueva_fitness;
    }

    void ejecutar() {
        string ffit = "../results/csv/fit_" + nombre_func + "_DE.csv";
        string fpob = "../results/csv/pob_" + nombre_func + "_DE.csv";

        ofstream f_fit(ffit); ofstream f_pob(fpob);
        f_fit << "Gen,MejorFitness\n"; f_pob << "Gen,X,Y,Fitness\n";

        inicializar();

        for (int i = 0; i < n_part; i++) f_pob << "0," << poblacion[i][0]
<< "," << poblacion[i][1] << "," << fitness_pob[i] << "\n";
```

```
        f_fit << "0," << fixed << setprecision(8) << gbest_val << "\n";


        for (int g = 1; g <= iters; g++) {

            evolucionar(g);

            f_fit << g << "," << fixed << setprecision(8) << gbest_val <<
"\n";


            if (g == iters / 2 || g == iters) {

                for (int i = 0; i < n_part; i++) {

                    f_pob << g << "," << poblacion[i][0] << "," << pobla-
cion[i][1] << "," << fitness_pob[i] << "\n";

                }

            }

        }

        f_fit.close(); f_pob.close();

        cout << "[" << nombre_func << "] Mejor: " << fixed << setpreci-
sion(8) << gbest_val << endl;

    }

};


int main() {

    system("mkdir -p ../results/csv ../results/plots");

    cout << "=== Ejecutando Evolucion Diferencial (DE) ===" << endl;


    // 1. Michalewicz

    vector<double> min_mich = {0.0, 0.0};

    vector<double> max_mich = {PI, PI};
```

```
    DE de_micha(50, 2, 200, min_mich, max_mich, michalewicz,
"Michalewicz");
    de_micha.ejecutar();


    // 2. Six-Hump Camel
    vector<double> min_camel = {-3.0, -2.0};
    vector<double> max_camel = {3.0, 2.0};
    DE de_camel(50, 2, 200, min_camel, max_camel, six_hump_camel, "Six-
HumpCamel");
    de_camel.ejecutar();


    cout << "Generando graficas en ../results/plots/ ..." << endl;
    system("python3 graficar_de.py");
    cout << "¡Proceso terminado exitosamente!" << endl;


    return 0;
}
```

Python (Gráficas) :

```
import pandas as pd
import matplotlib.pyplot as plt
import os
import warnings


warnings.filterwarnings("ignore")


CSV_DIR = "../results/csv/"
```

```
PLOT_DIR = "../results/plots/"


config = {
    "Michalewicz": {
        "opt": -1.8013,
        "coords": [(2.20, 1.57)],
        "lim_x": (0.0, 3.1416), "lim_y": (0.0, 3.1416)
    },
    "SixHumpCamel": {
        "opt": -1.0316,
        "coords": [(0.0898, -0.7126), (-0.0898, 0.7126)], # Tiene dos mín-
imos globales
        "lim_x": (-3.0, 3.0), "lim_y": (-2.0, 2.0)
    }
}


def graficar(func):
    fit_path = os.path.join(CSV_DIR, f"fit_{func}_DE.csv")
    pob_path = os.path.join(CSV_DIR, f"pob_{func}_DE.csv")


    if not os.path.exists(fit_path) or not os.path.exists(pob_path):
        print(f"No se encontraron los datos de {func}")
        return


    # --- Gráfica de Convergencia ---
    df_fit = pd.read_csv(fit_path)
    opt_teorico = config[func]["opt"]
```

```
    mejor_encontrado = df_fit['MejorFitness'].iloc[-1]


    plt.figure(figsize=(10, 5))
    plt.plot(df_fit['Gen'], df_fit['MejorFitness'], color='darkblue',
linewidth=2, label='Mejor Fitness (DE)')
    plt.axhline(y=opt_teorico, color='gray', linestyle=':', label=f'Optimo
Teorico ({opt_teorico})')


    plt.title(f'Convergencia DE - {func}')
    plt.xlabel('Generacion')
    plt.ylabel('Fitness')
    plt.grid(True, alpha=0.3)
    plt.legend()


    texto = (f"RESULTADOS\n"
             f"-------------------\n"
             f"Optimo teorico:\n{opt_teorico:.6f}\n\n"
             f"Valor encontrado:\n{mejor_encontrado:.6f}\n\n"
             f"Diferencia:\n{abs(opt_teorico - mejor_encontrado):.6f}\n\n"
             f"Gens: 200 | Parts: 50\nF: 0.8 | CR: 0.9")


    plt.subplots_adjust(right=0.75)
    plt.text(1.05, 0.5, texto, transform=plt.gca().transAxes,
             fontsize=10, verticalalignment='center',
             bbox=dict(boxstyle="round,pad=0.5", facecolor="white", edge-
color="darkblue"))
```

```
    plt.savefig(os.path.join(PLOT_DIR, f"Conv_{func}.png"), dpi=120,
bbox_inches='tight')

    plt.close()



    # --- Gráfica de Población ---

    df_pob = pd.read_csv(pob_path)

    gens = sorted(df_pob['Gen'].unique())

    titulos = ["Inicial (Gen 0)", "Intermedia", "Final"]


    fig, axes = plt.subplots(1, 3, figsize=(18, 5))


    for i, g in enumerate(gens):

        data = df_pob[df_pob['Gen'] == g]

        im = axes[i].scatter(data['X'], data['Y'], c=data['Fitness'],
cmap='viridis_r', s=40, edgecolor='gray')


        axes[i].set_title(f"Poblacion {titulos[i]} - {func}")

        axes[i].set_xlim(config[func]["lim_x"])

        axes[i].set_ylim(config[func]["lim_y"])

        axes[i].grid(True, alpha=0.2)


        # Marcar los mínimos globales teóricos

        for ox, oy in config[func]["coords"]:

            axes[i].scatter(ox, oy, marker='*', s=150, color='red', edge-
color='black', label='Optimo teorico' if i==0 and ox==config[func]["co-
ords"][0][0] else "")

        if i == 0:

            axes[i].legend(loc='best', fontsize=8)
```

```
    plt.subplots_adjust(right=0.9)
    cax = fig.add_axes([0.92, 0.15, 0.015, 0.7])
    fig.colorbar(im, cax=cax, label='Fitness')

    plt.savefig(os.path.join(PLOT_DIR, f"Pob_{func}.png"), dpi=120,
bbox_inches='tight')
    plt.close()

if __name__ == "__main__":
    graficar("Michalewicz")
    graficar("SixHumpCamel")
```

8. REFERENCIAS

1. Wikipedia https://es.wikipedia.org/wiki/Evoluci%C3%B3n_diferencial
2. https://www-sciencedirect-com.translate.goog/topics/computer-science/differential-evolution-algorithm?_x_tr_sl=en&_x_tr_tl=es&_x_tr_hl=es&_x_tr_pto=tc
3. Apuntes de clase